Reproducing and Extending PatchTST: Patch-Based Transformers for Long-Term Time Series Forecasting

Shi Yuan Chu, Ziming Gao, Wenhui Huang, Bing Wen Lim, Yuxiao Wang, Yanting Yang

1 Introduction

Long-term time series forecasting (LTSF) is important for energy scheduling, transportation, and finance. When Transformers are applied naively to LTSF, each time step becomes a token: attention costs $O(L^2)$ in sequence length L, and a single scalar carries little local context. Informer [2] reduces cost via sparse attention; Autoformer [3] adds seasonal–trend decomposition. Both still rely on point-wise time tokens. Zeng et al. [4] then showed that a one-layer linear forecaster (DLinear) can match or exceed strong Transformer baselines such as Autoformer on standard benchmarks, questioning where complexity should be spent.

PatchTST [5] instead tokenizes contiguous *patches* of length P, analogous to vision Transformers: each token summarizes a short subsequence (shape, not an isolated point) and the number of tokens drops roughly with stride S. Together with a Channel-Independent (CI) design—each of C variables is encoded as its own univariate series through a shared backbone—PatchTST gains regularization and effectively $C\times$ more training sequences. This project reproduces PatchTST from the paper in PyTorch, evaluates it together with DLinear and Autoformer on ETTh1 and three additional datasets, and studies a minimal cross-channel extension (ACCA, Section 5). Our contributions are:

- **Faithful reproduction.** We implement patching, attention, and the encoder as described in the paper, validate on ETTh1, and document four systematic mismatches between the paper and the official training code (Section 3).
- **Broader empirical evaluation.** Beyond ETTh1, we report Traffic, Beijing PM2.5, and a 42-pair FX dataset chosen for strong cross-channel correlation (Section 4).
- **ACCA and ablations.** We add a gated cross-channel mixer with attention/linear operators, pre-/post-head placement, and learned vs. forced gates; 14 runs quantify when mixing helps or hurts (Section 5).

2 PatchTST

2.1 Overview

PatchTST is an encoder-only Transformer for multivariate long-term forecasting. Given an input $\mathbf{X} \in \mathbb{R}^{L\times C}$ with lookback length L and C channels, the model predicts $\hat{\mathbf{Y}} \in \mathbb{R}^{H\times C}$ for forecast horizon H. The forward pass consists of seven stages: instance normalization, channel-independent reshaping, patching with embedding, a Transformer encoder, a linear prediction head, reshaping back to multivariate form, and denormalization.

2.2 Instance Normalization

Before entering the model, each sample is independently normalized along the time axis: the per-channel mean and standard deviation are computed, subtracted and divided respectively, and saved for later reversal:

$$\tilde{x}_{t,c} = \frac{x_{t,c} - \mu_c}{\sigma_c}, \qquad \mu_c = \frac{1}{L}\sum_{t=1}^{L} x_{t,c}, \qquad \sigma_c = \sqrt{\frac{1}{L}\sum_{t=1}^{L}(x_{t,c} - \mu_c)^2 + \epsilon} \tag{1}$$

This is separate from dataset-level z-score on the training split (Section 4): global scaling aligns channels, while instance norm removes each window's local level and variance so the model focuses on shape and copes with non-stationarity. The saved statistics are detached from the graph. The official repository swaps this for RevIN [6] with learnable affine parameters; the paper text describes the simpler form above, which we implement.

2.3 Channel Independence

The input $\mathbf{X} \in \mathbb{R}^{B \times L \times C}$ is reshaped to $\mathbb{R}^{(B \cdot C) \times L}$ so that each channel is an independent "sample." A single shared encoder is applied with *no* cross-channel attention inside the backbone. Benefits are regularization (harder to latch onto spurious multivariate correlations) and data efficiency (C times as many univariate trajectories per batch). A known tension is that channels with similar lookbacks but different futures can tug the shared weights in opposite directions; Section 5 tests whether a small gated mixer after the encoder mitigates that tension without redesigning PatchTST.

2.4 Patching

The univariate sequences are segmented into overlapping patches using a sliding window of length P with stride S. The number of patches is:

$$N = \left\lceil \frac{L - P}{S} \right\rceil + 1 \tag{2}$$

If the last window is incomplete, values are replicated to fill the patch. The paper's reference uses $N = \lfloor (L-P)/S \rfloor + 2$ with unconditional padding of S steps; when L divides cleanly this can introduce an extra synthetic patch. Our ceiling formula pads only when required and matches the reference whenever padding is actually needed. Attention complexity drops from $O(L^2)$ to $O(N^2)$; for L=336, P=16, S=8, we obtain N=41 tokens (≈8× fewer, ≈67× less quadratic work than full-length attention).

2.5 Patch Embedding

Each patch of P raw values is projected to a D-dimensional representation via a shared linear layer. A learnable positional encoding is added to each patch position, providing the model with ordering information that the permutation-invariant attention mechanism would otherwise lack:

$$\mathbf{z}_i = \mathrm{Dropout}(\mathbf{W}_{\mathrm{proj}}\, \mathbf{p}_i + \mathbf{e}_i) \tag{3}$$

where $\mathbf{e}_i$ is the i-th row of a learnable positional embedding matrix $\mathbf{E}_{\mathrm{pos}} \in \mathbb{R}^{N \times D}$.

2.6 Transformer Encoder

Each of M encoder layers applies (i) multi-head self-attention: linear projections produce queries, keys, and values, split into H heads over N patch tokens, then merged; (ii) a position-wise two-layer FFN with GELU, $D \to d_{ff} \to D$. Both blocks use residual connections followed

by **BatchNorm** in post-norm order, whereas the original Transformer uses LayerNorm [1]. Under CI the effective batch for normalization is $B{\cdot}C$, which helps BatchNorm statistics. The layer update is:

$$\mathbf{z}' = \mathrm{BatchNorm}(\mathbf{z} + \mathrm{Dropout}(\mathrm{MHA}(\mathbf{z}))) \tag{4}$$

$$\mathbf{z}'' = \mathrm{BatchNorm}(\mathbf{z}' + \mathrm{Dropout}(\mathrm{FFN}(\mathbf{z}'))) \tag{5}$$

2.7 Prediction Head and Denormalization

The encoder output $\mathbb{R}^{(B\cdot C)\times N\times D}$ is flattened to $\mathbb{R}^{(B\cdot C)\times(N\cdot D)}$ and projected to the prediction horizon H via a single linear layer with no hidden layers or activation function: $\hat{\mathbf{y}} = \mathbf{W}_{\text{head}}[\mathbf{z}_1;\ldots;\mathbf{z}_N] + \mathbf{b}$. The output is reshaped back to $\mathbb{R}^{B\times H\times C}$ and denormalized using the per-sample statistics saved during instance normalization, restoring the predictions to the original data scale.

3 Reproduction

We implemented PatchTST in PyTorch following the paper line-by-line and consulted the official repository [7] only where the text is ambiguous (e.g., dropout wiring). Core components—patching, multi-head attention, encoder stack—were written from scratch; minor details such as FFN dropout placement follow the reference. DLinear and Autoformer are taken from Time-Series-Library [8].

For validation we train all three models on ETTh1 (7 variables, hourly, 17,420 steps, standard 12/4/4 month split) with the authors' ETTh1 settings: lookback L=336, prediction length 96, patch length P=16, stride S=8, model dimension D=16, four heads, d_{ff}=128, M=3 layers, dropout 0.3, batch size 128, Adam with learning rate 10^{-4} (fixed for three epochs then multiplied by 0.9 each subsequent epoch, as in the reference script). We use random seed 42 and early-stopping patience 10 epochs instead of the paper's 2021 seed and patience 100, which slightly changes training trajectories but leaves the architecture unchanged.

We identified four reproducibility gaps between the paper and the reference code. **(1) Patching:** the reference always appends S padded steps, which can add a redundant patch when L aligns with the patch grid; we use the corrected rule in Section 2.4. **(2) Normalization:** the code uses RevIN rather than the paper's instance normalization; we implement the paper. **(3) Dropout rate:** Appendix text lists 0.2 for ETTh1 while the released script uses 0.3; we use 0.3 for comparability to the code. **(4) Attention dropout:** the reference applies dropout after the attention output projection and again on the residual branch, giving an effective rate of roughly $1-(1-0.3)^2 \approx 0.51$; the paper describes one dropout per sub-block, so we remove the redundant projection dropout.

Table 1: ETTh1 reproduction validation (pred_len = 96). Lower is better.

Model	MSE (Paper)	MSE (Ours)	MAE (Paper)	MAE (Ours)	Train Time
PatchTST	0.375	0.381	0.399	0.403	196s
DLinear	0.375	0.374	0.399	0.397	9s
Autoformer	0.435	0.528	0.446	0.491	1205s

Table 1 shows that our PatchTST and DLinear MSE/MAE are within about 2% of the numbers reported in Table 3 of [5] (PatchTST seed 42 and DLinear on ETTh1, horizon 96), which supports implementation correctness. Autoformer is farther from the published value because [5] reports the best result over six lookback lengths $\{24, 48, 96, 192, 336, 720\}$ whereas we evaluate only L=336.

4 Evaluation on Additional Datasets

To test whether PatchTST's edge over simpler baselines persists outside the original paper suite, we evaluate the same three model families on two public forecasting benchmarks and one FX panel. All runs use prediction horizon H=96 and reuse the ETTh1 architectural hyperparameters from Section 3 unless noted.

4.1 Datasets

Traffic [9] records hourly interstate traffic volume near Minneapolis together with weather and calendar-derived features (20 numeric channels). The forecasting target is `traffic_volume`. The series spans October 2012–September 2018 (48,204 hours).

Air quality [10] is the Beijing PM2.5 benchmark (41,757 hourly rows, 16 channels of pollution and meteorology). We predict the `pm2.5` concentration channel.

FX [11] contains 5,978 daily observations of 42 USD-quoted currency pairs. We forecast the Singapore dollar rate. High pairwise correlation makes this a deliberate stress test of strict CI: channels co-move through shared macro shocks.

Each dataset is split 70%/15%/15% chronologically into train, validation, and test.

4.2 Preprocessing

Splits preserve time order (no shuffling). Validation and test start only after a full lookback of L=336 steps is available. Z-score standardization uses training-set means and variances only, then applies the same scaling to validation and test to avoid leakage. Calendar features (hour, weekday, month, day-of-year) are mapped to $[-0.5, 0.5]$.

4.3 Results

On all three datasets, PatchTST and the ACCA-augmented variant in Table 2 outperform DLinear and Autoformer in MSE/MAE under our fixed configuration. Gains are largest over Autoformer, which tends to overfit or optimize slowly when data are limited relative to model capacity; DLinear remains a strong linear baseline but does not match the patch-based representation on these horizons.

Table 2: Results on additional datasets (pred_len = 96). Lower is better. All runs use the ETTh1 paper config.

Dataset	Model	MSE	MAE	Best Epoch	Train Time
Traffic	PatchTST	0.531	0.461	58	2962.6s
	PatchTST (ACCA)	0.530	0.460	58	3051.2s
	DLinear	0.581	0.517	15	20.5s
	Autoformer	0.685	0.539	10	526.3s
Air Quality	PatchTST	0.222	0.202	50	1668.3s
	PatchTST (ACCA)	0.221	0.201	50	1731.5s
	DLinear	0.278	0.289	11	11.7s
	Autoformer	0.404	0.337	9	429.9s
FX	PatchTST	0.089	0.185	88	934.2s
	PatchTST (ACCA)	0.089	0.185	88	981.6s
	DLinear	0.155	0.260	47	8.4s
	Autoformer	0.166	0.287	65	238.8s

5 Extension: Adaptive Cross-Channel Attention

5.1 Motivation and Architecture

The Channel-Independent (CI) design is a strong regularizer but is conceptually brittle when channels share partial—but not identical—temporal dynamics: two channels with similar look-backs that diverge in their horizons push contradictory gradients onto the shared backbone. The Foreign Exchange (FX) dataset is a controlled stress test for this hypothesis, since its 42 currency pairs share common macroeconomic drivers. Prior work has attacked the problem with a full cross-channel attention (Crossformer [12]) or an inverted attention axis (iTransformer [13]); both redesign the entire backbone. We ask a narrower, more surgical question: *can a minimal, gated cross-channel module recover that benefit while preserving the CI backbone?*

We insert an **Adaptive Cross-Channel Attention (ACCA)** module after the encoder without modifying it. Writing $\mathbf{Z} \in \mathbb{R}^{(B \cdot C) \times N \times D}$ for the per-channel representations, the module reshapes the batch to expose the channel axis, computes a cross-channel interaction $\mathbf{H}$, and blends the result back via a learned scalar gate $\mathbf{Z}' = \mathbf{Z} + \alpha(\mathbf{H} - \mathbf{Z})$ with $\alpha = \sigma(\alpha_{\text{raw}})$. We initialize $\alpha_{\text{raw}} = -4.6$ so $\alpha_0 \approx 0.01$: the model starts as a faithful copy of PatchTST and can open the cross-channel path only if it lowers the validation loss. Two mixing operators (`--acca_type`) are supported: a channel-axis multi-head self-attention with learnable channel embeddings (*attention*, $O(C^2 D)$ parameters), and a data-independent $C \times C$ linear mixing matrix (*linear*, $O(C^2)$ parameters). Two injection placements (`--acca_placement`) are supported: *pre-head* mixes the D-dim patch embeddings before the prediction head; *post-head* mixes the H-dim forecasts afterwards. Alongside the default learnable gate we support `--alpha_mode fixed_one`, which forces $\alpha = 1$ throughout training and makes the head see only the channel-mixed representation—the decisive ablation against "the gate is the reason ACCA is silent." All ACCA runs reuse the optimization / preprocessing pipeline of Sections 3–4 (paper ETTh1 config, Adam lr 10^{-4}, batch 128, patience 10, seed 42).

5.2 Main Results

Table 3 gives the main ACCA result. The learned gate converges to $\alpha_{\text{effective}} \in [0.0098, 0.0132]$ on every run—gradient descent actively chooses *not* to open the cross-channel path, even on FX. But the two variants are not equivalent: the linear mixer matches the CI baseline to $\pm 0.4\%$ MSE everywhere, whereas the parameter-heavier attention mixer is *consistently slightly worse* (+2.3% on Traffic, +10.1% on Air, +7.5% on FX; parity on ETTh1), apparently because its extra channel-conditioned parameters absorb enough gradient noise to perturb the backbone even at $\alpha \approx 0.01$.

5.3 Ablations

Table 4 breaks down placement, operator, and gate-mode on ETTh1 and FX. Pre-head and post-head are indistinguishable to three decimals on MSE on every dataset. The attention–linear gap on the larger-C datasets (0.006–0.022 MSE) cannot come from the mixing itself—the gate is almost closed—and instead comes from the $\approx 5\times$ more channel-conditioned parameters that the attention operator exposes to stochastic gradients. Forcing $\alpha = 1$ is decisive: linear `fixed_one` collapses +53.4% MSE on ETTh1 and +243.7% on FX, while the attention variant degrades +4.4% and +23.7%. The attention–linear asymmetry reflects architecture (the attention path's residual and LayerNorm leak most of the encoder signal through, whereas the linear mixer overwrites it); the ETTh1–FX asymmetry reflects channel count (the $C \times C$ mixer becomes a tighter bottleneck as C grows from 7 to 42). Both asymmetries point the same way: cross-channel mixing at this stage of PatchTST is *strictly harmful* information-theoretically, and the learned gate correctly identifies it as such.

Table 3: ACCA main-result table across four datasets (pred_len= 96, paper ETTh1 config, lr= 10^{-4}, batch size 128, patience 10, seed 42). `PatchTST` is the CI baseline from Section 4. α_{eff} is the final-epoch mean of $\sigma(\alpha_{\text{raw}})$. Lower MSE / MAE is better.

Dataset	Variant	MSE	MAE	Best Epoch	α_{eff}	ΔMSE (%)
ETTh1	PatchTST (CI)	0.381	0.403	37	–	0.0
	+ ACCA (linear, pre)	0.381	0.403	37	0.0103	+0.0
	+ ACCA (attn, pre)	0.381	0.403	47	0.0103	+0.0
Traffic	PatchTST (CI)	0.531	0.461	58	–	0.0
	+ ACCA (linear, pre)	0.530	0.460	58	0.0103	−0.1
	+ ACCA (attn, pre)	0.543	0.467	45	0.0117	+2.3
Air	PatchTST (CI)	0.222	0.202	50	–	0.0
	+ ACCA (linear, pre)	0.221	0.201	50	0.0102	−0.4
	+ ACCA (attn, pre)	0.244	0.231	39	0.0119	+10.1
FX	PatchTST (CI)	0.089	0.185	88	–	0.0
	+ ACCA (linear, pre)	0.089	0.185	88	0.0102	+0.2
	+ ACCA (attn, pre)	0.096	0.193	86	0.0098	+7.5

5.4 Discussion and Limitations

The ablations jointly rule out every trivial explanation for ACCA's behavior: the gate is not too conservative (`fixed_one` makes it much worse), the operator is not too weak (attention does not beat linear), the placement is not wrong (pre- and post-head agree), and the effect is not dataset-specific (it reproduces on all four). Under the CI training regime and horizon H=96, a late-stage cross-channel residual pathway is *net harmful*—not merely redundant. This strengthens the DLinear [4] observation that more capacity is not the limiting factor for long-horizon forecasting, and it contrasts with Crossformer [12], iTransformer [13], and TSMixer [14], all of which *do* get cross-channel gains—but only after redesigning the backbone so that channels sit on the token axis. The CI backbone of PatchTST evidently encodes a representation in which cross-channel structure is already implicit, so a surgical late-stage mixer has nothing to add. A second interpretation of the FX collapse is that strong correlation does not imply cross-variable *predictability*: currency pairs co-move because they share an unpredictable global driver, so mixing mostly propagates noise, and the damage therefore scales with C (+244% on FX vs. +53% on ETTh1).

Three limitations scope these conclusions. All runs use a single horizon (H=96); the CI-vs-mixing balance may shift when each channel's own past becomes less informative. ACCA is a local module and cannot replicate the token-axis reordering that backbone redesigns exploit; a head-to-head comparison with iTransformer under identical hyper-parameters is the natural next step. Finally, none of our datasets is constructed specifically to test cross-channel transfer (e.g. lead–lag), and such a dataset would provide a sharper probe than FX.

6 Conclusion

In summary, we rebuilt PatchTST in PyTorch and matched the published ETTh1 metrics for PatchTST and DLinear to within about 2% MSE (Table 1). Along the way we documented four concrete differences between the article and the official training code—patch padding, normalization choice, dropout hyper-parameters, and stacked attention dropout—and aligned our implementation with the paper's description (Section 3).

We further proposed and implemented the Adaptive Cross-Channel Attention (ACCA) mod-

Table 4: Combined placement and gate-mode ablation on the two diagnostic datasets (pred_len= 96, paper ETTh1 config). The *learned*-gate rows cover both placements; the `fixed_one` rows are pre-head only (post-head agrees to within optimizer noise).

Dataset	Operator	Placement	Gate	MSE	MAE	α_{eff}
ETTh1	linear	pre	learned	0.381	0.403	0.0103
	linear	post	learned	0.381	0.403	0.0103
	attn	pre	learned	0.381	0.403	0.0103
	attn	post	learned	0.381	0.403	0.0102
	linear	pre	fixed_one	0.585	0.522	1.000
	attn	pre	fixed_one	0.398	0.411	1.000
FX	linear	pre	learned	0.089	0.185	0.0102
	linear	post	learned	0.096	0.193	0.0101
	attn	pre	learned	0.096	0.193	0.0098
	attn	post	learned	0.096	0.193	0.0098
	linear	pre	fixed_one	0.306	0.367	1.000
	attn	pre	fixed_one	0.118	0.221	1.000

ule and ran a 14-run ablation across ETTh1, Traffic, Air, and FX varying the mixing operator, placement, and gate mode. The learned gate converges to $\alpha_{\text{effective}} \in [0.0098, 0.0132]$ on every run—actively choosing to stay closed. The linear variant ties the CI baseline to $\pm 0.4\%$ MSE everywhere; the attention variant, despite the nearly-closed gate, worsens MSE by $+2.3\%$ on Traffic, $+7.5\%$ on FX, and $+10.1\%$ on Air through gradient noise absorbed by its extra parameters. Forcing the gate open (`fixed_one`) catastrophically degrades both variants, with damage that scales with channel count ($+53.4\%$ on ETTh1 up to $+243.7\%$ on FX). These findings strengthen the original authors' CI claim—ACCA does not just fail to help, it can actively hurt—and indicate that a surgical late-stage mixer cannot substitute for the backbone redesigns of Crossformer [12], iTransformer [13], and TSMixer [14]. The final $\alpha_{\text{effective}}$ nevertheless serves as a cheap scalar diagnostic for whether a given dataset benefits from cross-channel context.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[2] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, pp. 11106–11115, May 2021, doi: 10.1609/aaai.v35i12.17325.

[3] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 22419–22430, 2021, doi: 10.48550/arXiv.2106.13008.

[4] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are Transformers Effective for Time Series Forecasting?" in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, pp. 11121–11128, Jun. 2023, doi: 10.1609/aaai.v37i9.26317.

[5] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers," in *International Conference on Learning Representations (ICLR)*, 2023, doi: 10.48550/arXiv.2211.14730.

[6] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, "Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: `https://openreview.net/forum?id=cGDAkQo1C0p`. [Accessed: Apr. 20, 2026]

[7] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "PatchTST: Official Implementation," GitHub, 2023. [Online]. Available: `https://github.com/yuqinie98/PatchTST`. [Accessed: Apr. 18, 2026]

[8] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis," in *International Conference on Learning Representations (ICLR)*, 2023, doi: 10.48550/arXiv.2210.02186. Code: `https://github.com/thuml/Time-Series-Library`. [Accessed: Apr. 18, 2026]

[9] J. Hogue, "Metro Interstate Traffic Volume," UCI Machine Learning Repository, 2019, doi: 10.24432/C5X60B. [Online]. Available: `https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume`. [Accessed: Apr. 20, 2026]

[10] X. Liang, T. Zou, B. Guo, S. Li, H. Zhang, S. Zhang, H. Huang, and S. X. Chen, "Assessing Beijing's PM2.5 Pollution: Severity, Weather Impact, APEC and Winter Heating," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 471, no. 2182, p. 20150257, 2015, doi: 10.1098/rspa.2015.0257. Dataset archived at UCI Machine Learning Repository, 2016. [Online]. Available: `https://archive.ics.uci.edu/dataset/381/beijing+pm2+5+data`. [Accessed: Apr. 20, 2026]

[11] thebasss, "Currency Exchange Rates," Kaggle, 2018. [Online]. Available: `https://www.kaggle.com/datasets/thebasss/currency-exchange-rates`. [Accessed: Apr. 20, 2026]

[12] Y. Zhang and J. Yan, "Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: `https://openreview.net/forum?id=vSVLM2j9eie`. [Accessed: Apr. 22, 2026]

[13] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, "iTransformer: Inverted Transformers Are Effective for Time Series Forecasting," in *International Conference on Learning Representations (ICLR)*, 2024, doi: 10.48550/arXiv.2310.06625.

[14] S.-A. Chen, C.-L. Li, N. Yoder, S. O. Arik, and T. Pfister, "TSMixer: An All-MLP Architecture for Time Series Forecasting," *Transactions on Machine Learning Research (TMLR)*, 2023, doi: 10.48550/arXiv.2303.06053.